

# UnifiedXRMotion: A Schema-Based Motion-Data Contract for Heterogeneous XR Avatar Pipelines

Anonymous Authors

Paper under double-blind review

**Abstract.** XR avatar-motion pipelines often entangle device-specific tracking APIs, skeletal layouts, validity semantics, motion processing, retargeting, and deployment decisions. This entanglement makes it difficult to reuse a pipeline across headsets, hand trackers, vision-based sources, and client-server execution contexts, even when each individual SDK exposes useful tracking data. We present UnifiedXRMotion, a Unity-based prototype that separates these concerns through a schema-based, validity-aware motion-data contract, a tracking-adapter boundary, and a local/remote node-placement abstraction. The contract represents skeletal motion as addressable motion slots defined by an application schema rather than by a fixed avatar rig. Each slot may carry optional pose components guarded by binary validity flags, allowing partial trackers to publish only the components they observe. In our prototype, the contract is instantiated with a 77-slot full-body schema, but the number of slots is an implementation example rather than the conceptual contribution. We describe how SDK-specific adapters publish into the contract, how event-driven motion nodes consume it, and how a network wrapper places selected nodes across client and server contexts. We do not claim new tracking, IK, retargeting, or input-to-photon latency measurement. Instead, we make a narrower systems contribution: a concrete, inspectable runtime contract for composing heterogeneous XR avatar-motion pipelines, together with case-study evidence from implemented adapters and application-level latency traces.

**Keywords:** Extended reality · Avatar motion · Tracking middleware · Motion-data contract · Runtime systems · Local/remote execution

## 1 Introduction

XR avatars depend on motion pipelines that span tracking acquisition, skeletal representation, motion processing, retargeting, and rendering. In practice these stages are frequently tied to one SDK or one avatar asset. A hand tracker may expose a vendor-specific joint order; a body tracker may report only a subset of joints; a vision model may provide positions but no reliable rotations; a retargeter may assume a particular rig hierarchy; and an offloaded node may require a serialized representation that differs from the in-process one. The result is not merely inconvenient engineering. It weakens reproducibility and makes it hard

to compare pipelines, replace trackers, or move computation between client and server without rewriting glue code.

Existing XR APIs and tracking middleware address important parts of this problem. OpenXR standardizes access to XR devices and extensions such as hand tracking [6, 7]. Earlier VR middleware, including VRPN [12], OpenTracker [10], and Ubiquitous Tracking [8], showed that device abstraction, dataflow, and distributed tracking are valuable architectural tools. These systems are direct prior art. UnifiedXRMotion does not claim novelty in generic middleware or graph execution. Its contribution is at the avatar-motion-pipeline level: a concrete runtime representation for partial skeletal motion, an adapter boundary for publishing device-specific observations into that representation, and a node-placement abstraction for local and remote execution contexts.

This paper is deliberately scoped. UnifiedXRMotion is not a new tracking algorithm, not a new inverse-kinematics solver, not a new avatar-retargeting method, and not a proof of universal device compatibility. It also is not a directed-acyclic-graph execution engine: the current runtime uses trigger-based scheduling and sender/receiver event propagation. The central design choice is to separate *slot identity* from *tracking availability*. A skeleton schema defines addressable motion slots and parent relations; a tracker publishes the subset it observes; each pose component is guarded by a validity flag; and downstream nodes consume only the components they require. In our Unity prototype, this contract is instantiated with a 77-slot full-body schema covering torso, limbs, head, and articulated hands. This number is not part of the contribution. It is one prototype schema instantiation. The same contract can be instantiated with smaller or larger schemas when adapters and consumers agree on the schema.

We make three contributions:

- **A schema-based, validity-aware motion-data contract** for XR skeletal motion, including addressable motion slots, parent relations, region views, optional pose components, binary validity flags, in-process buffer views, and validity-aware serialization.
- **An adapter boundary for heterogeneous tracking sources** that maps SDK-specific outputs into the shared contract. The prototype includes adapters for selected XR and vision-based sources, but we distinguish the reusable boundary from vendor-specific wrapper code.
- **A local/remote node-placement abstraction** that allows a motion-processing node to be wrapped for client-local, client-to-server, server-local, or server-to-client placement. We characterize existing application-level latency traces but do not claim input-to-photon measurement or general latency improvement.

## 2 Related Work

*Tracking middleware and runtime abstractions.* Device-independent VR tracking is not new. VRPN provides a device-independent and network-transparent

interface for VR peripherals [12]. OpenTracker introduced a modular software architecture for VR interaction and tracking configurations [10]. Ubiquitous Tracking modeled distributed spatial relationships for augmented reality as a graph of tracking transformations [8]. UnifiedXRMotion inherits the lesson that modularity and abstraction matter, but it targets a more specific layer: runtime skeletal motion contracts for avatar pipelines, where missing joints, partial component validity, region-specific views, retargeting consumers, and optional node placement must be handled explicitly.

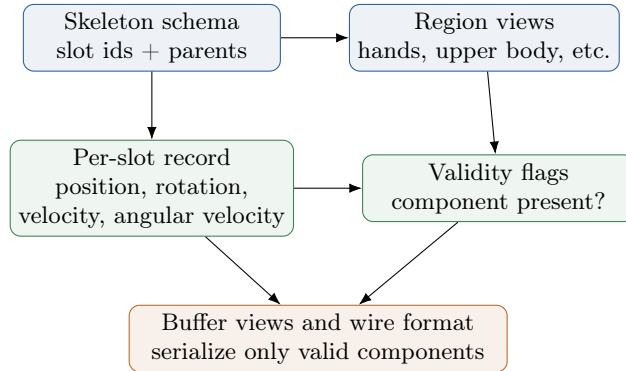
*XR APIs, avatar motion, and offloading.* OpenXR standardizes device access and extensions such as hand tracking [6, 7], while platform SDKs expose additional body, hand, controller, camera, or device-specific streams. These APIs tell an application how to access a source, but they do not prescribe how to represent partial skeletal motion across heterogeneous sources, mark unavailable components, or expose the same representation to filters, learned regressors, IK nodes, retargeters, and network serializers. Sparse avatar-motion reconstruction has advanced rapidly [5, 3, 14, 15, 11]. Avatar representation and retargeting affect embodiment and agency [16, 2, 13], with AvatarGo focusing on plug-and-play self-avatar setup [9]. UnifiedXRMotion does not compete with these algorithms. It provides infrastructure around them: a contract through which estimators, IK solvers, and retargeters can be represented as consumers and producers in the same runtime. For offloading, recent work such as XRgo investigates rendering offload for low-power XR devices [4]. UnifiedXRMotion’s local/remote placement abstraction is narrower: it makes node placement explicit and exposes application-level latency traces for motion-buffer transmission.

### 3 Schema-Based Motion-Data Contract

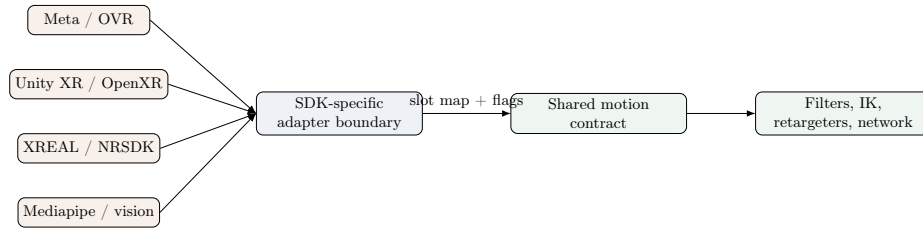
UnifiedXRMotion represents skeletal motion through a *schema* rather than through a hard-coded claim about the human body or avatar rig. A schema defines addressable motion slots, parent relations, and optional region views. A slot is an indexable location in the motion representation, such as a head, wrist, fingertip, tracker, prop, facial control, or application-specific joint. Each slot may carry pose components such as position, rotation, velocity, and angular velocity. Each component is guarded by a binary validity flag, so a tracker can publish only the data it observes and downstream nodes can avoid reading absent components.

Table 1 separates the general contract from the current prototype instantiation. This separation is essential. The prototype’s 77-slot layout is useful evidence that the contract can express a full-body avatar pipeline, but it is not a universal skeleton standard.

Partial tracking is the normal case in XR. A headset and controllers provide only a few rigid poses; a hand tracker may provide fingers but no lower body; a vision model may provide upper-body positions with unreliable rotations; and a full-body SDK may infer unobserved joints internally. UnifiedXRMotion therefore treats absence as first-class data rather than as an error. A downstream



**Fig. 1.** Motion-data contract. A schema defines addressable slots and parent relations; trackers publish partial pose records; binary validity flags identify which components are present; buffer views and serialization expose the same contract to runtime nodes and network transport.



**Fig. 2.** Adapter boundary. Native tracking sources remain SDK-specific, but each adapter maps observed data into the same schema-based motion contract.

node that needs rotations can reject or ignore slots whose rotation flag is absent; a filter that only needs positions can still operate when rotations are missing.

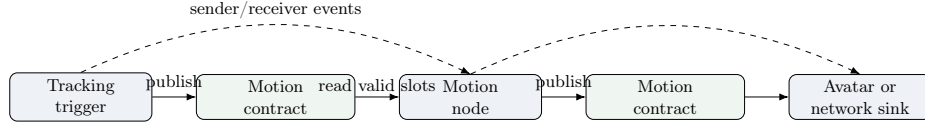
## 4 Adapter Boundary

UnifiedXRMotion uses SDK-specific adapters to map native tracking outputs into the motion-data contract. The central boundary is intentionally small: an adapter initializes the native source, polls or receives tracking data, exposes the slots it can publish, and returns pose records with validity flags. The adapter code is not the research novelty; much of it is necessarily vendor-specific wrapper code. The contribution is that all adapters target the same schema and validity semantics.

Table 2 preempts a likely reviewer objection. UnifiedXRMotion is partly an integration layer, because any practical XR system must interface with SDKs. The paper’s claim is not that the wrappers are novel. The claim is that a small adapter boundary plus a validity-aware contract prevents the rest of the motion

**Table 1.** Motion contract concepts. The prototype instantiation is evidence of implementation scope, not the conceptual boundary of the contract.

| Concept         | General definition                                              | Prototype instantiation                                                                | Limitation                                                                           |
|-----------------|-----------------------------------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Skeleton schema | Defines addressable motion slots and optional parent relations. | Full-body schema with 77 slots covering torso, limbs, head, and articulated hands.     | Schema agreement is required between adapters and consumers.                         |
| Region view     | Named subset of slots for partial-body processing.              | Views for hands, upper body, lower body, and full body.                                | Views are implementation-defined rather than discovered from arbitrary rigs.         |
| Pose record     | Optional components attached to a slot.                         | Position, rotation, velocity, and scalar angular velocity fields in a per-slot record. | Angular velocity is under-specified for full 3D rotation if represented as a scalar. |
| Validity        | Binary component-level availability.                            | Flags indicate whether position, rotation, velocity, or angular velocity is present.   | Continuous confidence is not preserved in the current contract.                      |
| Buffer view     | In-process access to preallocated records.                      | Kinematic buffer views expose whole-body or region-specific views.                     | Zero-copy applies only to in-process views, not network transport.                   |
| Wire format     | Serialization follows the same validity contract.               | Invalid components can be omitted during serialization.                                | Payload savings require a measured bytes-per-frame benchmark.                        |



**Fig. 3.** Actual runtime model. UnifiedXRMotion uses trigger-based scheduling and sender/receiver event propagation over a shared motion contract. It should not be interpreted as a general DAG execution engine.

pipeline from depending directly on each SDK’s joint order, validity semantics, and update conventions.

## 5 Runtime and Local/Remote Node Placement

The runtime is an event-driven motion-node pipeline. A trigger list identifies root nodes that poll or receive new motion data. When a root node updates, it invokes a sender event; downstream receivers subscribe to that event, read the upstream motion contract through buffer views, write their own output records, and publish another update. This design is intentionally simple and Unity-friendly. It should not be described as topological graph scheduling. There is no general cycle detector or graph optimizer in the current implementation.

For client-server deployments, UnifiedXRMotion provides a network wrapper around selected motion nodes. The wrapper exposes placement states describing where the producer, node execution, and consumer reside. The structural claim is that a node can be wrapped for different placements without rewriting the node

**Table 2.** Prototype adapter matrix. The table reports implementation coverage, not equal experimental validation.

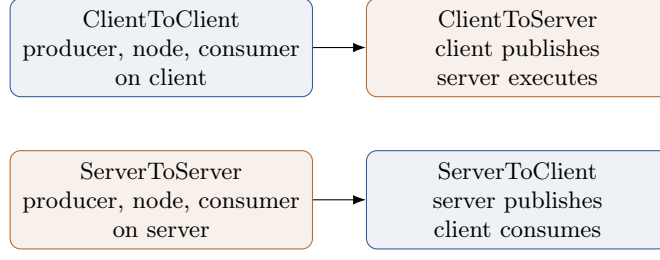
| Adapter/source                          | Native SDK or source                                                 | Motion coverage                     | Mapping into contract                                                     | Validation status and limitation                                                             |
|-----------------------------------------|----------------------------------------------------------------------|-------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Meta body/hand                          | OVR / OpenXR paths                                                   | Body, hands, HMD, controllers       | Native bone IDs and device poses are mapped to schema slots.              | Implemented and used in prototype traces; remains vendor-specific wrapper code.              |
| XREAL hand                              | NRSDK / XR Hands path                                                | Hands and head-relative context     | Hand joints are mapped to hand-region slots with vendor-specific offsets. | Implemented; previous video quality issues mean it should not support avatar-quality claims. |
| Unity XR                                | Unity XR / OpenXR input                                              | HMD, controller, hand sources       | Generic Unity poses are published into selected slots.                    | Implemented as adapter evidence; not a proof of full OpenXR coverage.                        |
| Vive / OpenXR                           | Vive OpenXR hand tracking                                            | Hand joints                         | Vendor hand joints are converted to hand-region slots.                    | Implemented; not evaluated as a separate user study condition.                               |
| Mediapipe                               | Vision-based landmarks                                               | Hands or upper body                 | Landmarks are converted into slot positions and binary validity.          | Implemented; continuous confidence is quantized away.                                        |
| Android AR / transform / replay / dummy | Mobile AR camera, scene transforms, recorded data, synthetic sources | Application-specific partial motion | Poses are published into selected slots for testing or replay.            | Useful for testing and demos; not hardware-generalization evidence.                          |

implementation. This is not a claim that all placements are deterministically equivalent or faster; equivalence and performance require separate measurement.

The current latency utility logs application-level motion-pipeline or RPC timing. It does not include sensor exposure, SDK internal sampling, Unity render submission, compositor scheduling, display scan-out, or photon emission. Table 3 defines the safe interpretation.

## 6 Implementation

The prototype is implemented as a Unity package. Its core implementation separates four concerns. First, motion slots are stored in preallocated buffers. Region views expose subsets of the schema, allowing a hand-only node, upper-body node, or full-body node to operate on the subset it needs. In-process view access avoids copying the entire pose record for each node. Network transmission necessarily serializes records and is not zero-copy. Second, SDK-specific adapters publish native tracking data into the shared contract. Some adapters are direct wrappers over vendor SDKs; others support synthetic transforms, replay, or vision-based sources. Third, the prototype includes IK, retargeting, codebook, filtering, record, and replay nodes. These nodes demonstrate that the contract



**Fig. 4.** Four placement states exposed by the local/remote node wrapper. The same node implementation can be wrapped for different execution contexts, but output equivalence and latency effects must be measured separately.

**Table 3.** Latency metric definition. The safe metric is useful for runtime debugging but must not be renamed as input-to-photon latency.

| Metric                                    | Included components                                                                                                                            | Excluded components                                                                                             | Safe interpretation                                                                              | Unsafe interpretation                                                                   |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Application-level motion-pipeline latency | Runtime tick/RPC delivery path between stamped motion-buffer send and receive events, plus framework scheduling effects included in that path. | Sensor sampling, native SDK buffering, clock synchronization, rendering, compositor, scan-out, photon emission. | Descriptive timing of the prototype’s motion-buffer path under specific local/remote placements. | Input-to-photon, motion-to-photon, or proof that offloading improves perceived latency. |
| Frame-time logging utility                | Unity frame interval or tick records exported for analysis.                                                                                    | Network transport decomposition and tracker capture time.                                                       | Runtime diagnostic signal.                                                                       | Network latency or user-perceived latency by itself.                                    |

can serve multiple consumers, but they are not claimed as new algorithms. Textbook solvers such as FABRIK [1] and codebook-style control inspired by prior work [11] should be cited as algorithmic background rather than framed as the paper’s novelty. Fourth, the implementation includes CSV and histogram export utilities for timing traces, plus record/replay utilities that can support reproducible evaluation. A stronger benchmark paper would require a released replay dataset, analysis scripts, and round-trip validation tests.

## 7 Evaluation and Case Study Evidence

The evidence in this medium paper is systems-oriented. We do not claim a complete user study of developer productivity, a vendor-pipeline baseline, or avatar-quality improvement. Instead, we report what the current implementation can substantiate: contract coverage, adapter diversity, original-vs-wrapper boundaries, and descriptive application-level latency traces.

**Table 4.** Original-vs-wrapper boundary. This table states what the paper can safely claim and what it must not claim.

| Component                                          | Classification                                       | Role in this paper           | Safe claim                                                         | Unsafe claim                                                          |
|----------------------------------------------------|------------------------------------------------------|------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------|
| Schema, slot records, validity flags, buffer views | Original framework logic                             | Motion-data contract         | Defines the runtime representation consumed by adapters and nodes. | Universal skeleton standard.                                          |
| Validity-aware serialization                       | Original logic over networking library               | Wire representation          | Can omit unavailable components according to flags.                | Measured payload reduction without a benchmark.                       |
| Tracking adapters                                  | SDK-specific wrappers plus mapping logic             | Evidence of adapter boundary | Map selected SDK outputs into the shared contract.                 | New tracking algorithms or universal device support.                  |
| Event-driven node interfaces                       | Original runtime logic                               | Composition model            | Supports sender/receiver motion-node pipelines.                    | Full DAG scheduler or graph optimizer.                                |
| IK, retargeting, codebook consumers                | Textbook, third-party, or application-specific nodes | Example consumers            | Demonstrate contract consumption by motion-processing nodes.       | Novel IK or retargeting contribution.                                 |
| Network wrapper                                    | Original logic over networking library               | Local/remote placement       | Provides four structural placement states for wrapped nodes.       | Deterministic equivalence or latency improvement without measurement. |
| Timing utilities                                   | Original support code                                | Descriptive characterization | Export application-level timing traces.                            | Input-to-photon or motion-to-photon measurement.                      |

Tables 1 and 2 show that the prototype contains the central elements needed for a reusable motion contract: schema-defined slots, region views, binary component validity, buffer views, SDK-specific adapters, and consumer nodes. The important result is not the number of slots or adapters; it is the separation of concerns. Device-specific logic is localized in adapters, while downstream nodes operate on the shared contract. This evidence supports “we implement a concrete contract and demonstrate it with selected sources.” It does not support “we prove generality across all XR devices.”

In a typical pipeline, an SDK adapter publishes partial tracking into the contract. A filter or estimator consumes the available components, a retargeter maps the resulting motion to an avatar rig, and an optional network wrapper changes the placement of selected nodes. This pipeline organization is the reusable part of the system. The retargeter itself is intentionally not claimed as new: it is a consumer of the representation.

**Table 5.** Descriptive application-level motion-pipeline latency traces from the prototype. Each condition currently contributes one continuous trace after warm-up and filtering, so the table should be read as characterization rather than an inferential comparison over repeated runs.

| Device       | Placement      | Mean  | SD   | p50   | p95   | p99   |
|--------------|----------------|-------|------|-------|-------|-------|
| Meta Quest 3 | Local          | 15.27 | 2.58 | 15.03 | 18.92 | 26.98 |
| Meta Quest 3 | Offloaded path | 13.89 | 0.44 | 13.88 | 14.52 | 14.88 |
| XREAL X4000  | Local          | 28.28 | 8.86 | 30.35 | 39.68 | 48.35 |
| XREAL X4000  | Offloaded path | 20.30 | 7.35 | 18.68 | 33.29 | 36.12 |

**Table 6.** Supported and unsupported claims after reframing.

| Claim                                                                  | Evidence                                                                  | Safe?  | Required fix if not safe                                                        |
|------------------------------------------------------------------------|---------------------------------------------------------------------------|--------|---------------------------------------------------------------------------------|
| UnifiedXRMotion defines a schema-based validity-aware motion contract. | Schema, slot records, validity flags, buffer views, serialization design. | Yes    | Add round-trip tests for stronger evidence.                                     |
| The prototype uses a 77-slot full-body schema.                         | Current Unity implementation.                                             | Yes    | Phrase only as prototype instantiation, not universal standard.                 |
| The adapter boundary localizes SDK-specific tracking logic.            | Implemented adapters mapped into shared records.                          | Mostly | Add LOC/diff-line table and one new-adapter case study.                         |
| The runtime is a DAG execution engine.                                 | Not supported by code audit.                                              | No     | Use event-driven motion-node pipeline wording.                                  |
| The network wrapper improves latency.                                  | Existing traces are single-condition descriptive data.                    | No     | Add repeated runs and vendor baselines; otherwise report characterization only. |
| The system measures input-to-photon latency.                           | Capture/render/compositor timestamps are absent.                          | No     | Add timestamping and synchronization instrumentation.                           |
| UnifiedXRMotion improves avatar quality.                               | No quantitative pose or perception evidence.                              | No     | Add pose-error, jitter, foot-skating, or user-perception study.                 |

## 7.1 Application-Level Latency Characterization

The existing prototype study collected latency traces for two devices and two execution modes. Reframed here, those traces are not evidence of input-to-photon performance. They are descriptive application-level traces of the motion-buffer path in a specific prototype configuration. Table 5 reports the retained summary statistics using that interpretation.

The traces suggest that placement can reshape the application-level distribution, but they do not identify the cause. The current instrumentation does not isolate serialization time, network transport, SDK buffering, scheduler effects, or device temperature. Therefore, the safe claim is that UnifiedXRMotion exposes timing data for its own motion-buffer path, not that offloading generally improves XR latency.

## 8 Discussion

*Why a motion contract paper is defensible.* The previous framing as a graph framework invites the wrong comparison and overstates the implementation. The defensible research object is the contract. In avatar-motion pipelines, the hard engineering problem is not only acquiring tracking data; it is keeping track of what a source actually observed, representing that partial data consistently, and making downstream nodes robust to missing components. UnifiedXRMotion addresses that problem through schema-defined slots and component-level validity.

*What the abstraction enables.* The contract lets adapters, filters, retargeters, and network serializers share a common representation. It also makes missing data explicit. This is valuable for XR because different trackers expose different body regions and component reliability. A hand source, a sparse controller source, a replay stream, and a vision source can all publish partial motion into the same runtime representation, provided that they agree on the schema.

*Where the abstraction is still weak.* The contract is not yet a stable standard. It lacks mandatory timestamps, coordinate-frame metadata, continuous confidence, and schema versioning. These omissions matter. Without timestamps, latency and synchronization claims are limited. Without coordinate-frame metadata, frame conversions remain adapter-specific and easy to misconfigure. Without continuous confidence, vision trackers lose information when confidence is collapsed into binary validity. Without schema versioning, external tools cannot safely exchange saved motion streams across versions.

*Local/remote placement remains structural.* The network wrapper is a useful systems abstraction, but structural reuse is not the same as equivalence. A wrapped node may receive data at different times, serialize different subsets, or run under different scheduling conditions. A future version should run the same recorded motion stream through local and remote placements and report per-slot output differences.

## 9 Limitations

This work has significant limitations. First, it does not measure input-to-photon or motion-to-photon latency. The reported traces exclude sensor sampling, SDK internal processing, compositor scheduling, rendering, display scan-out, and photon emission. Second, the prototype does not prove universal device support. It implements selected adapters, but each new tracker still requires a schema mapping, coordinate conversion, and validation. Third, the current prototype schema is one implementation example; arbitrary avatar rigs require schema agreement and retargeting between the runtime representation and the avatar-specific skeleton. Fourth, UnifiedXRMotion does not introduce new tracking, IK,

or retargeting algorithms. The included solvers and retargeters are representative consumers of the contract. Fifth, the current paper lacks a controlled adapter-authoring study, a vendor-native latency baseline, a bytes-per-frame wire-format benchmark, and a deterministic local/remote equivalence check. These are necessary before making stronger claims about developer productivity, transport efficiency, or placement performance.

## 10 Conclusion

UnifiedXRMotion reframes heterogeneous XR avatar-motion integration as a contract problem. Rather than proposing a universal skeleton, a new tracker, or a DAG execution engine, it defines a schema-based validity-aware representation in which trackers publish partial motion into addressable slots and downstream nodes consume only valid components. The prototype demonstrates this contract through selected SDK adapters, event-driven motion nodes, representative retargeting and IK consumers, and a local/remote node-placement wrapper. The current evidence is not sufficient for broad latency or avatar-quality claims, but it is sufficient to support a narrower EuroXR contribution: a concrete runtime contract and abstraction boundary for composing heterogeneous XR avatar-motion pipelines.

## References

1. Aristidou, A., Lasenby, J.: FABRIK: A fast, iterative solver for the inverse kinematics problem. *Graphical Models* **73**(5), 243–260 (2011)
2. Cheymol, A., Fribourg, R., Lécuyer, A., Normand, J.M., Argelaguet, F.: Beyond my real body: Characterization, impacts, applications and perspectives of “dissimilar” avatars in virtual reality. *IEEE Transactions on Visualization and Computer Graphics* **29**(11), 4426–4437 (2023)
3. Dai, P., Zhang, Y., Liu, T., Fan, Z., Du, T., Su, Z., Zheng, X., Li, Z.: HMD-Poser: On-device real-time human motion tracking from scalable sparse observations. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 874–884 (2024)
4. Gao, S., Liu, J., Jiang, Q., Sinclair, F., Sentosa, W., Godfrey, P.B., Adve, S.V.: XRgo: Design and evaluation of rendering offload for low-power extended reality devices. In: *Proceedings of the ACM Multimedia Systems Conference* (2025)
5. Jiang, J., Streli, P., Qiu, H., Fender, A., Laich, L., Snape, P., Holz, C.: AvatarPoser: Articulated full-body pose tracking from sparse motion sensing. In: *Computer Vision – ECCV 2022. Lecture Notes in Computer Science*, vol. 13665, pp. 443–460. Springer (2022). [https://doi.org/10.1007/978-3-031-20065-6\\_26](https://doi.org/10.1007/978-3-031-20065-6_26), [https://link.springer.com/chapter/10.1007/978-3-031-20065-6\\_26](https://link.springer.com/chapter/10.1007/978-3-031-20065-6_26)
6. Khronos Group: Khronos releases OpenXR 1.0 specification. <https://www.khronos.org/news/permalink/khronos-releases-openxr-1.0-specification> (2019), press release
7. Khronos Group: XR\_EXT\_hand\_tracking. [https://registry.khronos.org/OpenXR/specs/1.1/man/html/XR\\_EXT\\_hand\\_tracking.html](https://registry.khronos.org/OpenXR/specs/1.1/man/html/XR_EXT_hand_tracking.html) (2025)

8. Newman, J., Wagner, M., Bauer, M., MacWilliams, A., Pintaric, T., Beyer, D., Pustka, D., Strasser, F., Schmalstieg, D., Klinker, G.: Ubiquitous tracking for augmented reality. In: *Proceedings of the 3rd IEEE and ACM International Symposium on Mixed and Augmented Reality*. pp. 192–201. IEEE Computer Society (2004)
9. Ponton, J.L., Monclús, E., Pelechano, N.: AvatarGo: Plug and play self-avatars for VR. In: *Eurographics 2022 – Short Papers*. pp. 77–80. The Eurographics Association (2022). <https://doi.org/10.2312/egs.20221037>
10. Reitmayr, G., Schmalstieg, D.: An open software architecture for virtual reality interaction. In: *Proceedings of the ACM Symposium on Virtual Reality Software and Technology*. ACM (2001)
11. Starke, S., Starke, P., He, N., Komura, T., Ye, Y.: Categorical codebook matching for embodied character controllers. *ACM Transactions on Graphics* **43**(4), 142:1–142:14 (2024). <https://doi.org/10.1145/3658209>
12. Taylor, R.M., Hudson, T.C., Seeger, A., Weber, H., Juliano, J., Helser, A.T.: VRPN: A device-independent, network-transparent VR peripheral system. In: *Proceedings of the ACM Symposium on Virtual Reality Software and Technology*. ACM (2001)
13. Weidner, F., Boettcher, G., Arevalo Arboleda, S., Diao, C., Sinani, L., Kunert, C., Gerhardt, C., Broll, W., Raake, A.: A systematic review on the visualization of avatars and agents in AR & VR displayed using head-mounted displays. *IEEE Transactions on Visualization and Computer Graphics* **29**(5), 2596–2606 (2023). <https://doi.org/10.1109/TVCG.2023.3247072>
14. Xu, V., Gao, C., Hoffmann, H., Ahuja, K.: MobilePoser: Real-time full-body pose estimation and 3D human translation from imus in mobile consumer devices. In: *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST)*. pp. 1–11 (2024)
15. Yang, D., Kang, J., Ma, L., Greer, J.D., Ye, Y., Lee, S.H.: DivaTrack: Diverse bodies and motions from acceleration-enhanced three-point trackers. *Computer Graphics Forum* **43**(2) (2024). <https://doi.org/10.1111/cgf.15057>
16. Yun, H., Ponton, J.L., Andujar, C., Pelechano, N.: Animation fidelity in self-avatars: Impact on user performance and sense of agency. *IEEE Transactions on Visualization and Computer Graphics* (2023)